Lab03

This lab has three parts:

- **Part 1** walks you through examples on how to use R to run a two-sample t-test, calculate the pooled standard deviation and standard error of the difference; in addition, this part also provides guidance on how to use R to obtain quantiles of the T-distribution (a handy way to finding the information without using tables)
- **Part 2** explains how to conduct inference on difference in means from paired data;
- **Part 3** provides self-assessment opportunities for the two topics in this lab.

#1.1 Using R to obtain T-quantiles

This provides a short description on how to use the R function to obtain the value for a desired quantile form the T-distribution.

Example 1: what is the 0.975 quantile of a 20 df T distribution?

```
qt(0.975, 20)
```

```
[1] 2.085963
```

In the command above, the first argument is the desired quantile while the second is the degrees of freedom.

Example 2: what is the 97.5 percentile of a 20 df T distribution?

```
qt(0.975, 20)
```

```
[1] 2.085963
```

Please note that because qt() requires a proportion, you must convert a percentile (0 - 100) to a proportion (0 - 1)

Example 3: What is the lower tail probability for T = -2.1 (i.e., P[T <= -2.1]) when the T distribution has 60 df?

```
pt(-2.1, 60)
```

```
[1] 0.01997088
```

Example 4: What is the upper tail probability for T = 2.1 (i.e., P[T >= 2.1]) when the T distribution has 60 df?

```
pt(2.1, 60, lower=F)
```

```
[1] 0.01997088
```

Note: neither of these is the two-sided *p*-value. If you don't see what you need to do to compute the p-value, please don't use the pt() function.

#1.2 Two sample (pooled) *t*-tools

This example uses the same data set on Creativity from Lab02. We will start by reading in the data (recall to set up the path identifying your working directory):

```
# read in the data
library(tidyverse) # make sure to install if it isn't already
creativity <- read.csv("creativity.csv", header=TRUE)

# and print the top entries as check for the data set
head(creativity)
```

```
  treatment score
1 intrinsic  12.0
2 intrinsic  12.0
3 intrinsic  12.9
4 intrinsic  13.6
5 intrinsic  16.6
6 intrinsic  17.2
```

#1.2.1 Perform a two-sample *t*-test in R.

We use the following command:

```
t.test(score~treatment, data=creativity, var.equal=T)
```

```
    Two Sample t-test

data:  score by treatment
t = -2.8837, df = 45, p-value = 0.00601
alternative hypothesis: true difference in means between group extrinsic and group intrinsic
95 percent confidence interval:
 -6.964878 -1.236571
sample estimates:
mean in group extrinsic mean in group intrinsic
               15.78261                19.88333
```

A *formula* in R is a function which offers a convenient way to specify the response and grouping variables. Here is a brief explanation on how this particular formula works:

- The response variable is on the left-hand side (in this case, it is called *score*)
- The **grouping** variable is on the right-hand side (in this case it is called *treatment*).
- The part of the formula *data=creativity* specifies the data frame with the variables names in the formula.
- The *var.equal=T* argument specifies that the function should assume that the two groups have the same variance and to use the pooled standard deviation in the *t*-test. Although equal variance is the assumption made throughout Module 2, the default in R for the t.test function is the unequal variance (Welch or Welch-Satterthwaite) t-test.
- The output from the t-test function includes:
  - two-sided *p*-value: p-value = (You know it is two-sided because of the next line, which tells you that the alternative is not equal to 0)
  - error df and the t-statistic
  - 95% confidence interval for the difference of means
  - the sample averages (estimated means) for each group.

Additional notes:

- It is good practice to check that the degrees of freedom (top of output, df=) match what you expect. If not, the data weren't read or interpreted correctly.
- To get something other than a 95% confidence interval, specify the desired coverage as a decimal fraction in the conf.level= argument. For example,

```
t.test(score ~ treatment, data=creativity, var.equal=T, conf.level=0.90)
```

```
    Two Sample t-test

data:  score by treatment
t = -2.8837, df = 45, p-value = 0.00601
alternative hypothesis: true difference in means between group extrinsic and group intrinsic
90 percent confidence interval:
 -6.488952 -1.712497
sample estimates:
mean in group extrinsic mean in group intrinsic
               15.78261                19.88333
```

will provide a 90% confidence interval for the difference.

#1.2.2 Compute the pooled sd and the se of the difference

The t.test() function **does not** report the pooled sd or the se of the difference. You will find below an example demonstrating how to compute these quantities from a data set. The code assumes that there are only two levels of the treatment variable (and it is set up to check this condition).

The code provided here is written in the same way as the code for a randomization test. You copy treatment and response information into two specifically-named variables. The rest of the code can be ran without changes. *Aside: Those who know about R functions will see that this code could easily be converted into a function.*

I start by copying the treatment information into the variable named group and the response information into the variable names response.

```
group <- creativity$treatment
response <- creativity$score
```

The rest of the code can be run without modification.

```
if (length(unique(group)) != 2) {
  stop(paste("group does not have two levels.  I see:\n",
    paste(unique(group), collapse=' ')) )
}
n <- table(group)
#   table counts how many of each unique value, want to use the treatment

s <- tapply(response, group, sd)
#  sd of each group, first argument is the response, 2nd is the treatment
#    and we want to use the sd() function which returns the sd of the group

# both n and sd are vectors with 2 values

pooled.sd <- sqrt(sum( (n-1)*s^2)/( sum(n) - 2) )
cat('pooled sd\n')
pooled.sd

se.diff <- pooled.sd * sqrt(1/n[1] + 1/n[2])

# or you could use

se.diff <- pooled.sd * sqrt(sum(1/n))
cat('se of the difference in means\n')
se.diff
```

```
pooled sd
[1] 4.873435
se of the difference in means
[1] 1.422049
```

The following explanations are only relevant if you want to know some useful R tricks.

- **if ...**

  Counts the number of unique values in group. If not 2, stop and write out an error message that includes the names of the groups.

- **n <- table(group)**

  Count the number of observations in each group and save in a vector called n. This will have 2 values, one for each group.

- **s <- tapply(response, group, sd)**

  Calculate the within-group sd for each group. The arguments to tapply are the variable to be worked on, the variable de ning groups, and the function to apply to all the values in each group. This call computes the sd for each group. Again, the result is a vector with 2 values.

- **pooled.sd <- ...**

  Compute the pooled sd. R allows math on vectors. So **(n-1)*s^2** results in a vector of 2 values, with each being the sample size (n) times the variance (s^2) for each group. **sum()** adds those. The denominator is the total sample size, **sum(n)** minus 2 df.

- **se.diff <- ...**

  Computes the standard error of the difference, either by working separately on each sample size, or by using the vector math equivalent.

#2. Paired *t*-tools

We will use here the Schizophrenic twin data set.

```
schiz <- read.table('case0202.txt', header=T, as.is=T)
head(schiz)
```

```
  unaff  aff
1  1.94 1.27
2  1.44 1.63
3  1.56 1.47
4  1.58 1.39
5  2.06 1.93
6  1.66 1.26
```

Calculate the difference:

```
schiz$diff <- schiz$unaff - schiz$aff
```

Remember that "<-" assigns a result (right hand piece) to the specified variable (left hand piece). If that variable is in a data frame (e.g., schiz$diff), the data frame gains a variable. We can see the effect of the assignment by looking at the first 6 observations of the new version of the data frame.

Calculating the standard error:

As you already know, there is not a built-in function to compute the standard error for observations stored in a vector. We will use the function we discussed in Lab02 to compute this quantity.

```
# function se(): calculates standard error
# x: the data
se <- function(x) {
    s <- sd(x, na.rm=TRUE)  # computes sample standard deviation
    n <- sum(!is.na(x))     # computes number of non-missing observations
    s/sqrt(n)               # computes and returns standard error
}
```

Notes on missing values: If the result of an R function applied to a vector is NA, that means the data set includes missing values. R's missing value code is NA. If you attempt to calculate a mean, median, variance, or standard deviation of data that includes a missing value, the default behavior is to return a result of NA. If this happens to you, figure out why the data set includes missing values.

Also note that there are ways to tell R to ignore missing values. It's safer to figure out why values are missing.

Applying the se() function to the series of differences, we obtain:

```
se(schiz$diff)
```

```
[1] 0.06152713
```

There are two ways to calculate the paired t-test. One is to compute the difference (as we did above), then do a one-sample t-test:

#Paired t-test as a one-sample t-test:

```
t.test(schiz$diff)
```

```
    One Sample t-test

data:  schiz$diff
t = 3.2289, df = 14, p-value = 0.006062
alternative hypothesis: true mean is not equal to 0
```

```
95 percent confidence interval:
 0.0667041 0.3306292
sample estimates:
mean of x
0.1986667
```

You can also specify the two variables to t.test() and specify that the observations are paired (by using paired=T). The results are exactly the same as the one-sample results.

#Paired t-test as a two paired values

```
t.test(schiz$unaff, schiz$aff, paired=T)
```

```
    Paired t-test

data:  schiz$unaff and schiz$aff
t = 3.2289, df = 14, p-value = 0.006062
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 0.0667041 0.3306292
sample estimates:
mean difference
      0.1986667
```

The next line of code is another way of expressing the same R commands, in an alternative way using the function with():

```
with(schiz, t.test(unaff, aff, paired=T))
```

```
    Paired t-test

data:  unaff and aff
t = 3.2289, df = 14, p-value = 0.006062
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 0.0667041 0.3306292
sample estimates:
mean difference
      0.1986667
```

#3. Self-Assessment

3.1. **Pooled *t-tools:*** This example will make use of the same tomato data used in Module 1. As a reminder, these data are from a randomized experiment comparing a proposed "better" fertilizer to the usual fertilizer. Seven plants (treatment a) were randomly assigned to the "better" fertilizer; the other seven plants received the usual fertilizer. The measurement units are pounds of ripe tomato. Analyze the data using a two-sample t-test.

a) What is the pooled standard deviation?

b) How many degrees of freedom are associated with that pooled sd?

c) What is the estimated difference, expressed as better (trt a) – usual (trt b)? Include units.

d) What is the standard error of the difference?

Note/reminder, unless stated otherwise, this will be based on the pooled sd.

e) What is the two-sided p-value?

f) Write a one-sentence interpretation of that p-value.

ANSWERS:

a) 6.18

b) 12

Note: If you got 6.49, or something close to that, you did an unequal variance (Welch) t-test.

c) 6.53 lbs. (6.5314 before rounding)

Note: most programs give you b-a = -6.53. Better – usual is a-b. Always be aware which version makes sense for your question.

d) 3.30 lbs

e) 0.0716

Note: If you got 0.092, you did an unequal variance (Welch) t-test.

f) Weak evidence of a difference between the better and usual fertilizers. (Or, if you really want to use 5% as a cutoff, no evidence …).

Two incorrect interpretations are: no difference between the two fertilizers; the two fertilizers have the same average yield.

3.2. **Paired t-tools:**

The data in Darwin.csv are classic data collected by Charles Darwin. He was interested in the effect of cross pollination in plants. He collected seeds from flowers on a single plant that were

self-pollinated (pollen from that plant) or cross-pollinated (pollen from a distant plant). He then planted these seeds in pots, grew the plants and measured their height, in inches. Two seeds were planted in each pot.One was cross-pollinated seed; the other was self-pollinated.

a) Are these data two independent samples or paired? Explain your choice.

b) How many pots were used in this study?

c) Estimate the mean difference, as cross – self pollinated.

d) Calculate the standard error of that difference.

e) Use a t-test to test the null hypothesis of no difference in mean height between cross- and self-pollinated seeds. Report the p-value and a short conclusion.

f) Estimate a 95% confidence interval for the difference.

ANSWERS:

a) These data are paired because two seeds, one from each treatment, were planted in each pot. A cross-pollinated observation is "linked" to the self-pollinated observation in the same pot.

b) 15

Look at the number of rows of data

c) 2.62 inches.

If you wanted a sentence describing the mean difference, you could write something like: Cross-pollinated seeds produce plants that are 2.62 inches taller than the plants from self-pollinated seeds.

d) 1.22 inches.

Calculated as the sd / sqrt(n), if not reported by your software. t.test() in R doesn't automatically report this, so you have to "hand" calculate it, although you use R to calculate the sd, which is the tedious bit.

e) p-value = 0.0498. Evidence of a difference in average height between cross- and self-pollinated seeds (p = 0.0498).

6) (0.0032, 5.23)